

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Aryan Arun	Reg. No.:	192411157
Section / Batch:	BE CSE / 2024	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I _____ Aryan Arun (192411157) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Aryan Arun _____

RUBRICS FOR CALCULATION

Comparative Analysis

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CSA0624 – DESIGN AND ANALYSIS OF ALGORITHMS

Comparative Analysis of Classic Algorithm Pairs

DP vs Greedy • Dijkstra vs Bellman-Ford • Prim vs Kruskal • Coin Change

NAME: ARYAN ARUN
REG.NO: 192411157

1. Knapsack Comparison: 0/1 Knapsack (DP) vs Fractional Knapsack (Greedy)

Instance: weights = [10, 20, 30], values = [60, 100, 120], capacity $W = 50$.

1.1 Setting Up the Items

Item	Weight	Value	Value / Weight Ratio
Item 1	10	60	6.0
Item 2	20	100	5.0
Item 3	30	120	4.0

1.2 Solving with 0/1 Knapsack (Dynamic Programming)

Each item must be taken whole or not at all. Build $dp[i][w]$ = best value using the first i items with capacity w , using the recurrence $dp[i][w] = \max(dp[i-1][w], dp[i-1][w - \text{weight}_i] + \text{value}_i)$.

Evaluating all capacity-50 combinations directly is quick for three items:

Combination	Total Weight	Total Value	Feasible (≤ 50)?
Item 1 only	10	60	Yes
Item 3 only	30	120	Yes
Item 1 + Item 2	30	160	Yes
Item 1 + Item 3	40	180	Yes
Item 2 + Item 3	50	220	Yes
Item 1 + Item 2 + Item 3	60	300	No — exceeds capacity

0/1 Knapsack result: take Item 2 + Item 3 (weight $20 + 30 = 50$, value $100 + 120 = 220$). This is confirmed by the full DP table, where $dp[3][50] = 220$.

1.3 Solving with Fractional Knapsack (Greedy)

Greedy sorts items by value/weight ratio and fills the sack from the highest ratio down, allowing fractions of an item.

Step	Item Taken	Amount	Weight Used	Value Gained	Remaining Capacity
1	Item 1 (ratio 6.0)	100% (whole item)	10	60	40
2	Item 2 (ratio 5.0)	100% (whole item)	20	100	20
3	Item 3 (ratio 4.0)	$20/30 = 66.7\%$ (fraction)	20	80	0

Fractional Knapsack result: take all of Item 1, all of Item 2, and $2/3$ of Item 3. Total weight = $10 + 20 + 20 = 50$, total value = $60 + 100 + 80 = 240$.

1.4 Comparing the Two Solutions

Aspect	0/1 Knapsack (DP)	Fractional Knapsack (Greedy)
Items selected	Item 2 + Item 3 (whole)	Item 1 + Item 2 (whole) + 2/3 of Item 3
Total value achieved	220	240
Optimal for its own rules?	Yes — DP guarantees the optimum over all 0/1 subsets	Yes — greedy by ratio is provably optimal when fractions are allowed
Time complexity	$O(n \cdot W)$ — pseudo-polynomial	$O(n \log n)$ for the sort, then $O(n)$
Space complexity	$O(n \cdot W)$, reducible to $O(W)$	$O(1)$ beyond sorting

The two answers (220 vs 240) are not a sign that one algorithm is 'wrong' — they are solving different problems. Fractional Knapsack is allowed to slice Item 3, capturing extra value that 0/1 Knapsack is forbidden from taking because it must keep every item intact.

1.5 Strengths and Limitations

0/1 Knapsack (DP)

- Strength: always finds the true optimum when items are indivisible (the realistic case for physical objects, jobs, or discrete investments).
- Strength: the recurrence generalizes to many constrained variants (multiple knapsacks, item counts, dependencies).
- Limitation: pseudo-polynomial time — cost grows with the capacity value W , not just n , so it becomes expensive for very large or non-integer capacities.

Fractional Knapsack (Greedy)

- Strength: extremely fast, $O(n \log n)$, and simple to implement and reason about.
- Strength: provably optimal, but only under the fractional model.
- Limitation: the greedy choice (highest ratio first) does not carry over to the 0/1 model — it can lock in a locally attractive item that blocks a better combination, so applying it to indivisible items only gives an approximation, not a guarantee.

1.6 When Greedy Is Optimal vs When DP Is Necessary

- Greedy is optimal whenever the problem has both a greedy-choice property and optimal substructure that survives fractional/continuous choices — Fractional Knapsack, activity selection, and Huffman coding are classic examples.
- DP is necessary whenever choices are indivisible or interact with each other so that a locally best choice can foreclose a globally better one — 0/1 Knapsack, Coin Change with arbitrary denominations (see Section 4), and edit distance are classic examples.
- A practical litmus test: if you can prove an exchange argument showing the greedy choice is always contained in some optimal solution, greedy is safe; if you cannot, default to DP.

1.7 Insights on Method Selection

Use Fractional Knapsack when the resource is genuinely divisible (liquids, budgets, time slices, bandwidth) — the greedy solution is both optimal and far cheaper to compute. Use 0/1 Knapsack DP when items are atomic (selecting projects, packing discrete cargo, choosing investments with fixed sizes) — greedy's speed is not worth an answer that can be meaningfully suboptimal. In this instance, the 240 vs 220 gap (about 9%) illustrates that the cost of misapplying greedy to an indivisible problem is not negligible and grows with problem size.

2. Shortest Path Comparison: Dijkstra vs Bellman-Ford

Instance (contains a negative edge, no negative cycle): vertices $\{A, B, C, D\}$, directed edges $A \rightarrow B = 1$, $A \rightarrow D = 2$, $D \rightarrow C = 1$, $C \rightarrow B = -10$. Source vertex: A.

2.1 Applying Dijkstra's Algorithm

Dijkstra repeatedly pops the unvisited vertex with the smallest tentative distance and finalizes it, assuming that distance can never improve later.

Step	Vertex Popped	Action	Tentative Distances After Step
1	A (dist 0)	Relax $A \rightarrow B = 1$, $A \rightarrow D = 2$	$A=0$, $B=1$, $D=2$, $C=\infty$
2	B (dist 1)	Finalize B. No outgoing edges relax anything new.	B finalized at 1
3	D (dist 2)	Relax $D \rightarrow C$: $2+1=3$	$C=3$
4	C (dist 3)	Relax $C \rightarrow B$: $3+(-10) = -7$, but B is already finalized, so Dijkstra ignores this improvement.	C finalized at 3

Dijkstra's reported result: $A=0$, $B=1$, $D=2$, $C=3$ — this is WRONG for B. Once Dijkstra finalizes a vertex it never revisits it, so the negative edge $C \rightarrow B$, discovered too late, can never correct B's distance.

2.2 Applying Bellman-Ford's Algorithm

Bellman-Ford relaxes every edge in the graph, repeated for $(|V| - 1)$ rounds, allowing distances to improve at any point regardless of visit order.

Round	Edge Relaxations Applied	Updated Distances
1	$A \rightarrow B$: $0+1=1$ (update B). $A \rightarrow D$: $0+2=2$ (update D). $D \rightarrow C$: $2+1=3$ (update C). $C \rightarrow B$: $3+(-10)=-7$ (update B).	$A=0$, $B=-7$, $D=2$, $C=3$
2 & 3	Re-checking all edges finds no further improvement.	$A=0$, $B=-7$, $D=2$, $C=3$ (converged)

Bellman-Ford's result: $A=0$, $B=-7$, $D=2$, $C=3$ — this is CORRECT. The true shortest path to B is $A \rightarrow D \rightarrow C \rightarrow B$ with cost $2+1-10 = -7$, cheaper than the direct edge $A \rightarrow B = 1$.

2.3 Why Dijkstra Fails Here

Dijkstra's correctness proof relies on the invariant that once a vertex is popped with the smallest tentative distance among all unvisited vertices, no future relaxation can ever improve it — which is only true if every edge weight is non-negative. With non-negative weights, any alternate path through an unvisited vertex can only be longer, never shorter. A negative edge breaks this: a path that looked long while passing through several small positive edges can suddenly become cheap after one negative edge is added, but by the time that edge is discovered, Dijkstra has already 'locked in' the earlier, worse answer for the vertex it points to. That is exactly what happens to B above — it is finalized at 1 before the algorithm ever learns about the -7 route through C.

2.4 Correctness, Complexity, and Limitations

Aspect	Dijkstra	Bellman-Ford
Correct with negative edges?	No — can produce wrong distances silently	Yes, as long as there is no negative-weight cycle reachable from the source
Negative cycle detection	Not supported	Yes — an (V) -th relaxation round that still finds an improvement signals a negative cycle
Time complexity	$O((V + E) \log V)$ with a binary heap	$O(V \cdot E)$
Typical use case	Large graphs with non-negative weights (road networks, routing)	Graphs that may contain negative weights (currency arbitrage, constraint graphs) or that need cycle detection

2.5 Performance Insight and Algorithm Selection

On this 4-vertex graph the runtime difference is irrelevant, but it illustrates the general trade-off: Dijkstra's heap-based approach is asymptotically faster and is the right default whenever weights are guaranteed non-negative. Bellman-Ford trades that speed for robustness — it tolerates negative edges and can detect negative cycles, which Dijkstra cannot do at all (it would either loop or return a silently wrong answer). The decision rule is simple: if the graph can contain negative weights or must be checked for negative cycles, use Bellman-Ford; otherwise, use Dijkstra for its better asymptotic performance on large graphs.

3. MST Comparison: Prim's vs Kruskal's Algorithm

Instance: vertices {A, B, C, D, E}, weighted undirected edges:

Edge	Weight
B–C	1
A–B	2
A–C	3
B–D	4
C–D	5
C–E	6
D–E	7

3.1 Constructing the MST with Kruskal's Algorithm

Kruskal sorts all edges by weight and greedily adds each edge that does not create a cycle (checked via union-find), regardless of which vertices it touches.

Order	Edge Considered	Weight	Decision	Reason
1	B–C	1	Add	Connects two separate components {B}, {C}
2	A–B	2	Add	Connects {A} to {B, C}
3	A–C	3	Reject	A and C already connected — would form a cycle
4	B–D	4	Add	Connects {D} to {A, B, C}
5	C–D	5	Reject	C and D already connected — would form a cycle
6	C–E	6	Add	Connects {E} to the rest — MST complete (4 edges, 5 vertices)

Kruskal's MST: {B–C, A–B, B–D, C–E}, total weight = $1 + 2 + 4 + 6 = 13$.

3.2 Constructing the MST with Prim's Algorithm

Prim grows a single tree from a start vertex, at each step adding the cheapest edge that connects a vertex already in the tree to one that is not. Starting at A:

Step	Tree So Far	Candidate Edges from Tree	Edge Chosen	New Tree
1	{A}	A–B (2), A–C (3)	A–B (2)	{A, B}
2	{A, B}	A–C (3), B–C (1), B–D (4)	B–C (1)	{A, B, C}
3	{A, B, C}	C–D (5), B–D (4), C–E (6)	B–D (4)	{A, B, C, D}

Step	Tree So Far	Candidate Edges from Tree	Edge Chosen	New Tree
4	{A, B, C, D}	D–E (7), C–E (6)	C–E (6)	{A, B, C, D, E} — done

Prim's MST: {A–B, B–C, B–D, C–E}, total weight = 2 + 1 + 4 + 6 = 13.

3.3 Comparing Edge Selection Strategies

Aspect	Prim's Algorithm	Kruskal's Algorithm
Selection strategy	Grows one connected tree outward, always picking the cheapest edge leaving the current tree	Globally sorts all edges once and greedily adds the cheapest edge that avoids a cycle, anywhere in the graph
Intermediate structure	Always a single connected tree	A forest of multiple growing components until they merge
Data structure used	Min-priority queue over frontier edges/vertices	Union-Find (disjoint set) plus a sorted edge list
Result on this instance	{A-B, B-C, B-D, C-E}, weight 13	{B-C, A-B, B-D, C-E}, weight 13

Both algorithms select the same 4 edges here and arrive at the same total weight of 13. This is expected: because all edge weights in this instance are distinct, the MST is unique, so any correct algorithm must converge on it — only the order of discovery differs (Kruskal reaches B–C first because it is the cheapest edge globally; Prim reaches it second because it must first touch a vertex already in its tree).

3.4 Performance in Dense vs Sparse Graphs

Aspect	Prim's Algorithm	Kruskal's Algorithm
Time complexity	$O(E \log V)$ with a binary heap, $O(E + V \log V)$ with a Fibonacci heap	$O(E \log E)$, dominated by sorting the edge list
Best suited to	Dense graphs (E close to V^2), since it never needs to look at the full edge list at once	Sparse graphs (E close to V), since sorting a short edge list is cheap and union-find operations are nearly constant time
Memory access pattern	Needs adjacency-list/matrix access to the current frontier	Needs the full edge list sorted up front

3.5 Implementation Differences and Conclusions

- Prim's is naturally suited to adjacency-matrix or adjacency-list representations and is easy to adapt when the graph is discovered incrementally (e.g., streaming or implicit graphs).
- Kruskal's needs the entire edge set available up front to sort it, and relies on an efficient union-find with path compression and union by rank to keep cycle checks near $O(1)$ amortized.
- Both are greedy algorithms that are provably correct because of the Cut Property and Cycle Property of MSTs — the difference is purely which greedy order they exploit (vertex-frontier growth vs. globally cheapest edge first), not the correctness guarantee.
- Practical rule: choose Prim's for dense graphs or when the graph arrives incrementally; choose Kruskal's for sparse graphs or when edges are already available as a list.

4. Coin Change Problem: Greedy vs Dynamic Programming

Instance: coin denominations = {1, 3, 4}, target amount = 6.

4.1 Solving with the Greedy Approach

Greedy always picks the largest denomination that does not exceed the remaining amount.

Step	Remaining Amount	Coin Picked	Amount After
1	6	4 (largest ≤ 6)	2
2	2	1 (largest ≤ 2 , since 3 and 4 don't fit)	1
3	1	1	0

Greedy result: $4 + 1 + 1 = 6$, using 3 coins.

4.2 Solving with Dynamic Programming

Let $dp[a]$ = minimum number of coins to make amount a , with $dp[a] = 1 + \min(dp[a-1], dp[a-3], dp[a-4])$ over valid choices, $dp[0] = 0$.

Amount	0	1	2	3	4	5	6
$dp[\text{amount}]$	0	1	2	1	1	2	2
Coins used to build it	—	{1}	{1,1}	{3}	{4}	{4,1} or {1,4}	{3,3}

DP result: $dp[6] = 2$, achieved with $3 + 3 = 6$, using only 2 coins.

4.3 Comparing the Solutions

Aspect	Greedy	Dynamic Programming
Coins used	$4 + 1 + 1$ (3 coins)	$3 + 3$ (2 coins)
Optimal?	No — uses one more coin than necessary	Yes — DP explores every combination via the recurrence and guarantees the minimum
Time complexity	$O(\text{amount})$ or $O(\text{number of denominations})$ — very fast	$O(\text{amount} \times \text{number of denominations})$
Space complexity	$O(1)$	$O(\text{amount})$

4.4 Why Greedy Fails Here

Greedy's largest-first rule only works when the coin system is 'canonical' — informally, when every denomination is a clean multiple relationship of the ones below it (like 1, 5, 10, 25 in US currency), so that taking the biggest coin never blocks a better combination later. The set {1, 3, 4} is not canonical: taking the 4 immediately for amount 6 leaves 2, which can only be paid with two 1-coins, whereas skipping the 4 and using two 3-coins covers the exact amount in one fewer coin. Greedy has no mechanism to look ahead and see that its 'best available' choice at each step creates a worse remainder — it commits irrevocably, while DP evaluates all possible remainders and keeps only the genuinely optimal one.

4.5 Insights on Choosing the Appropriate Method

- Use greedy coin change only when the denomination set is known to be canonical (verified either by proof or by exhaustive checking for small amounts) — most real-world currency systems qualify, which is why greedy 'feels' correct from everyday experience.
- Use DP whenever the denomination set is arbitrary, unknown to be canonical, or supplied as input by a user/program — DP is always correct regardless of the coin set, at the cost of higher time and space complexity.
- This instance is a compact counter-example that should be kept in mind whenever someone assumes greedy 'obviously' works for coin change: with coins $\{1, 3, 4\}$ and amount 6, greedy overshoots the optimum by 50% (3 coins instead of 2).

5. Cross-Cutting Insights: Greedy vs DP vs Specialized Graph Algorithms

Pair	Do the strategies ever disagree?	Root cause when they disagree
0/1 Knapsack (DP) vs Fractional Knapsack (Greedy)	Yes, by design — they solve different problem variants	Greedy's advantage (splitting items) is unavailable to the 0/1 model
Dijkstra vs Bellman-Ford	Yes, when negative edges exist	Dijkstra's finalize-and-never-revisit invariant breaks under negative weights
Prim vs Kruskal	No, when edge weights are distinct — both find the same unique MST	Both rely on the same Cut/Cycle Property, just applied in a different order
Coin Change: Greedy vs DP	Yes, when the denomination set is non-canonical	Greedy's locally best pick can create an unfavorable remainder it cannot revisit

A common thread runs through all four comparisons: greedy algorithms are fast and simple because they commit to a choice and never look back, which is only safe when the problem has a structural guarantee — an exchange argument, a matroid property, or non-negative weights — that the greedy choice cannot be improved on later. Dynamic programming (and its graph analogue, Bellman-Ford's repeated relaxation) is slower but keeps every option open until the recurrence forces a provably minimal choice, which is why it remains correct on the harder, less-structured instances where greedy silently gives it up.

Practical selection guide: reach for the greedy method first, but only after confirming (by proof, known theory, or a citable property of the problem) that the greedy-choice property holds for your exact input constraints. If that confirmation is not available, or the instance is adversarial/arbitrary (as in Coin Change with $\{1,3,4\}$, or graphs with negative weights), fall back to the DP or exhaustive-relaxation approach even though it costs more time and memory — correctness should not be traded for speed without a guarantee.